✓ MODULE 1: Python Basics & Environment Setup

Module Objectives

By the end of this module, students will be able to:

- Understand the core features and advantages of Python.
- Set up Python and related tools on their systems.
- Write basic Python programs using correct syntax.
- Work with variables, data types, and operators.
- Use input/output statements for user interaction.

Detailed Concepts

1. Introduction to Python

- **What is Python?**
  - High-level, interpreted, general-purpose programming language.
  - Developed by **Guido van Rossum** in the late 1980s.
- **Key Features:**
  - Easy to learn and read (simple syntax).
  - Dynamically typed.
    - A language is called **dynamically typed** when **you don't need to declare the data type of a variable** — the type is automatically determined **at runtime** (when the program runs), not before.
  - Interpreted language.
    - When we say that **Python is an *interpreted language***, we mean that **Python code is executed line by line by an interpreter** instead of being compiled into machine code all at once before running.
  - Large standard library.
    - Python comes with a **built-in collection of modules and packages**, called the **standard library**, which provides **many useful tools and functions** — so you don't need to write everything from scratch or install extra tools for common tasks.
  - Portable across platforms (Windows, Mac, Linux).
    - **Portability** in programming means that **you can write a Python program on one operating system (e.g., Windows) and run it on another (e.g., Mac or Linux) with little or no changes.**
  - Extensive third-party libraries.
    - Python has thousands of ready-made tools and packages created by other developers (not part of the core language) that you can use in your own programs. These are called *third-party libraries*.
- **Popular Applications:**
  - Web development, data science, machine learning, scripting, automation, software testing, etc

Basic Syntax and Structure

- **Whitespace and Indentation:**
  - Indentation is mandatory in Python (typically 4 spaces).
- **Comments:**
  - Single line: `# This is a comment`
  - Multi-line: triple quotes `''' or """`

Example Program:

```
        # This program prints a message
        message = "Welcome to Python!"
print(message)
```

Variables and Data Types

- **Variable Declaration:**

```
x = 10
name = "Alice"
pi = 3.14
```

- **Rules for Variable Names:**
  - Must start with a letter or underscore
  - Case-sensitive
  - Cannot use reserved keywords

1. Assigning Multiple Variables in One Line

Python lets you assign values to multiple variables simultaneously in a very clean and readable way.

Why use this?

- Saves lines of code.
- Makes initialization easier and more organized.

Syntax:

```
var1, var2, var3 = value1, value2, value3
```

Example:

```
x, y, z = 10, 20, 30
print(x)  # 10
print(y)  # 20
print(z)  # 30
```

Important:

- The number of variables on the left **must match** the number of values on the right.

```
a, b = 1, 2, 3  # ✗ Error: too many values to unpack
```

2. Assigning the Same Value to Multiple Variables

Sometimes you want several variables to have the **same initial value**.

Syntax:

```
a = b = c = value
```

Example:

```
a = b = c = 100
print(a)  # 100
print(b)  # 100
print(c)  # 100
```

Explanation:

- This means all three variables point to the same value.

Important:

- For **immutable types** like integers or strings, this is safe.
- For **mutable types** (like lists), this means the variables refer to the **same object** — changing one changes all!

3. Swapping Variables Without a Temporary Variable

Python supports elegant variable swapping using tuple unpacking.

Traditional swapping in other languages:

```
temp = a
a = b
b = temp
```

Python way:

```
a, b = b, a
```

Example:

```
a = 5
b = 10
a, b = b, a
print(a)  # 10
print(b)  # 5
```

Explanation:

- The right side `(b, a)` creates a tuple.
- The tuple is unpacked into variables on the left.
- This is concise, readable, and efficient.

Data Types in Python

Definition:

A **data type** in Python defines the kind of value a variable can hold — such as numbers, text, lists, etc.
Python is **dynamically typed**, so you don't declare data types — Python figures them out based on the value.

1. Numeric Types

Type	Description	Example
`int`	Integer numbers	`x = 10`
`float`	Decimal numbers	`x = 3.14`
`complex`	Complex numbers (real + imag)	`x = 2 + 3j`

```
a = 10       # int
b = 3.14     # float
c = 2 + 3j   # complex
```

2. Text Type

Type	Description	Example
`str`	String (text)	`x = "Python"`

```
name = "Alice"
```
You can use single `' '`, double `" "`, or triple quotes `''' '''`.

3. Sequence Types

Type	Description	Example
`list`	Ordered, changeable, allows duplicates	`[1, 2, 3]`
`tuple`	Ordered, **unchangeable**, allows duplicates	`(1, 2, 3)`
`range`	Sequence of numbers	`range(1, 10)`

```
fruits = ["apple", "banana", "cherry"]  # list
colors = ("red", "green", "blue")       # tuple
nums = range(5)                         # range object
```

4. Mapping Type

Type	Description	Example
`dict`	Key-value pairs, unordered (Python 3.7+ keeps order)	`{"name": "Alice", "age": 25}`

```
student = {"name": "John", "age": 22, "grade": "A"}
```

5. Set Types

Type	Description	Example
`set`	Unordered, no duplicates	`{1, 2, 3}`
`frozenset`	Immutable set	`frozenset({1, 2})`

```
myset = {1, 2, 3, 2}          # Output: {1, 2, 3}
```

6. Boolean Type

Type	Description	Example
`bool`	Only two values: `True` or `False`	`x = True`

```
is_valid = True
is_empty = False
```

7. Binary Types

Type	Description	Example
`bytes`	Immutable byte data	`b"Hello"`
`bytearray`	Mutable byte data	`bytearray(5)`
`memoryview`	View of byte data	`memoryview(b"abc")`

These are more advanced — used for binary file handling or networking.

How to Check a Variable's Type?

```
x = 5
print(type(x))    # Output: <class 'int'>
```

Python List Data Type —

What is a List?

A **list** in Python is a **collection of items** stored in a single variable.
It can hold **multiple values** of **any data type** — numbers, strings, booleans, or even other lists.

```
example = [10, "hello", 3.14, True]
```

Key Features of List

Feature	Explanation
Ordered	Items have a fixed position (index starting at 0)
Mutable	You can change, add, or remove items
Duplicates	Allowed
Mixed data types	Yes (e.g., int, str, float in same list)

Creating a List

```
empty = []                              # empty list
numbers = [1, 2, 3, 4, 5]
names = ["Ali", "Sara", "John"]
mixed = [42, "yes", 2.5, False]
```

Accessing Elements

```
fruits = ["apple", "banana", "cherry"]

print(fruits[0])     # apple
print(fruits[-1])    # cherry (last item)
```

Common List Methods (With Examples)

Function	Description	Example	Output
`append(x)`	Adds item at end	`l.append(100)`	[10, 20, 100]
`insert(i, x)`	Inserts at index	`l.insert(1, 5)`	[10, 5, 20]
`extend(iter)`	Adds all items	`l.extend([30,40])`	[10, 20, 30, 40]
`remove(x)`	Removes first `x`	`l.remove(10)`	[20, 30]
`pop(i)`	Removes by index	`l.pop(0)`	Returns 10
`clear()`	Empties list	`l.clear()`	`[]`
`index(x)`	Finds index	`l.index(20)`	`1`
`count(x)`	Counts `x`	`l.count(20)`	`2`
`sort()`	Sorts ascending	`l.sort()`	[1, 2, 3]
`reverse()`	Reverses list	`l.reverse()`	[3, 2, 1]
`copy()`	Copies list	`new = l.copy()`	Same as `l`

Built-in Functions with Lists

Function	Description	Example	Output
`len()`	Length of list	`len(l)`	`5`
`sum()`	Sum of items	`sum([1,2,3])`	`6`
`min()`	Smallest item	`min([1,2,3])`	`1`
`max()`	Largest item	`max([1,2,3])`	`3`
`sorted()`	Returns new sorted list	`sorted([3,1,2])`	`[1,2,3]`
`any()`	True if any item is True	`any([0, 0, 1])`	`True`
`all()`	True if all items are True	`all([1, 2, 3])`	`True`

Indexing & Slicing

```
letters = ['a', 'b', 'c', 'd', 'e']

print(letters[1:4])   # ['b', 'c', 'd']
print(letters[-3:-1]) # ['c', 'd']
```

Real-life Example

```
shopping = ["milk", "bread", "eggs"]

shopping.append("butter")
shopping.remove("bread")
```

```
print(shopping[0])  # milk
```

Input and Output

Input from User:

```
name = input("Enter your name: ")
print("Hello, " + name)
```

Converting Input:

```
age = int(input("Enter age: "))
print("You will be", age + 1, "next year.")
```

Operators

- **Arithmetic Operators**: `+`, `-`, `*`, `/`, `//`, `%`, `**`
- **Relational/Comparison**: `==`, `!=`, `>`, `<`, `>=`, `<=`
- **Logical Operators**: `and`, `or`, `not`
- **Assignment Operators**: `=`, `+=`, `-=`, `*=`, etc.

Example:

```
x = 10
y = 3
print(x + y)      # 13
print(x // y)     # 3 (floor division)
```

Common Mistakes to Avoid

- Forgetting to indent properly.
- Misunderstanding dynamic typing (e.g., mixing types).
    - This refers to **errors that happen when you assume Python will handle type conversions automatically**, or **you try to mix incompatible data types without converting them properly.**
    - Since Python is **dynamically typed**, you don't declare variable types — **but it's still your job to use them correctly**.
    - **EXAMPLE:**
        - age = 25 # int
        - message = "Age is: " + age # ✗ Error: can't add str and int

- Not converting `input()` to `int`/`float` before performing arithmetic.
    - When you get user input using `input()`, it's treated as text. To do math, you **must convert that text to a number type** (`int` or `float`).
    - EXAMPLE:

```
num1 = input("Enter first number: ")  # User types 5
num2 = input("Enter second number: ") # User types 10

result = num1 + num2
print("Result:", result)
```

- Using reserved keywords as variable names.

Tuple Data Type

What is a Tuple?

A **tuple** in Python is a collection used to store **multiple items** in a **single variable**, just like a list.
BUT: Tuples are **immutable**, meaning you **cannot change**, **add**, or **remove** elements after creation.

```
example = (10, "hello", 3.14, True)
```

Key Features of Tuple

Feature	Explanation
Ordered	Items have a fixed index (starting from 0)
Immutable	You **cannot** modify the tuple after creation
Allows Duplicates	Yes
Can store mixed data types	Yes (e.g., int, str, float in one tuple)

Creating a Tuple

```
empty = ()
one_item = (10,)               # Note the comma!
numbers = (1, 2, 3, 4, 5)
names = ("Ali", "Sara", "John")
mixed = (42, "yes", 2.5, False)
```

Accessing Elements

```
fruits = ("apple", "banana", "cherry")

print(fruits[0])     # apple
print(fruits[-1])    # cherry (last item)
```

You CANNOT:

- Add items: `tuple.append()` → Not allowed
- Remove items: `tuple.remove()` → Not allowed
- Change items: `tuple[0] = 'orange'` → Error

You CAN:

- **Count elements**
- **Find index**
- **Loop through**
- **Convert to list and back**
- **Slice** tuples (like strings/lists)

Tuple Methods (Only Two!)

Method	Description	Example	Output
`count(x)`	Counts number of times `x` appears	`t.count(10)`	2
`index(x)`	Returns index of first `x`	`t.index(10)`	0

```
t = (10, 20, 10, 30)
print(t.count(10))   # 2
print(t.index(30))   # 3
```

Slicing Tuples

```
t = (0, 1, 2, 3, 4, 5)

print(t[1:4])     # (1, 2, 3)
print(t[-3:])     # (3, 4, 5)
```

Convert Tuple to List (to make changes)

```
t = (10, 20, 30)
lst = list(t)
lst.append(40)
t = tuple(lst)
print(t)  # (10, 20, 30, 40)
```

Tuple vs List

Feature	List (`[]`)	Tuple (`()`)
Mutable	Yes	No
Performance	Slower	Faster
Methods	Many	Few
Use Case	When data changes	When data should not change

Real-Life Example

```
months = ("Jan", "Feb", "Mar", "Apr")

# Use for fixed data (like month names or days)
print(months[0])  # Jan
```

Built-in Functions That Work on Tuples

These are general Python functions, not tuple methods:

Function	Description	Example	Output
`len()`	Length of tuple	`len(t)`	4
`max()`	Maximum element	`max((2,5,8))`	8
`min()`	Minimum element	`min((2,5,8))`	2
`sum()`	Sum of all items	`sum((2,5,8))`	15
`sorted()`	Returns sorted list	`sorted((3,1,2))`	[1, 2, 3]
`any()`	True if any item is true	`any((0, 0, 1))`	True
`all()`	True if all items are true	`all((1, 2, 3))`	True

Tuple Unpacking

What is Tuple Unpacking?

Tuple Unpacking refers to the process of assigning the values of a tuple (or any iterable) directly to a corresponding number of variables in a single line.

Basic Syntax:

```
tuple_name = (value1, value2, value3)
a, b, c = tuple_name
```

- Each variable on the left gets the corresponding value from the tuple on the right.

Example 1: Basic Tuple Unpacking

```
person = ("Ali", 25, "Lahore")

name, age, city = person

print(name)  # Ali
print(age)   # 25
print(city)  # Lahore
```

Important Rule:

The number of variables on the left **must match** the number of elements in the tuple.

```
a, b = (1, 2, 3)  # ✕ Error: too many values to unpack
```

Used With Other Iterables Too:

Tuple unpacking works with:

- Lists
- Strings
- Sets (but order is not guaranteed)
- Any iterable object

Example with List:

```
data = [10, 20]
x, y = data
print(x, y)  # 10 20
```

Example 2: Tuple Unpacking in a Loop

Useful when iterating over a list of tuples.

```
students = [("Ali", 90), ("Sara", 85), ("John", 78)]

for name, marks in students:
    print(f"{name} scored {marks}")
```

Example 3: Ignoring Values using _ (underscore)

If you don't need all the values, use _ as a placeholder.

```
person = ("Ali", 25, "Lahore")
name, _, city = person
print(name, city)  # Ali Lahore
```

Extended Unpacking (Python 3+)

You can use * to capture "the rest" of the values.

Example:

```
a, *b = (1, 2, 3, 4, 5)
print(a)  # 1
print(b)  # [2, 3, 4, 5]

a, *b, c = (1, 2, 3, 4, 5)
print(a)  # 1
print(b)  # [2, 3, 4]
print(c)  # 5
```

Tuple Unpacking with `enumerate()`:

```
names = ["Ali", "Sara", "John"]

for index, name in enumerate(names):
    print(f"{index}: {name}")
```

When is Tuple Unpacking Useful?

Use Case	Description
Looping over list of tuples	Extract multiple values directly
Returning multiple values from a function	Assign them easily
Working with `enumerate()`	Get both index and value
Swapping variables	Cleaner and faster syntax

Swapping Values Example:

```
a = 10
b = 20
a, b = b, a
print(a, b)  # 20 10
```

Common Mistake:

```
x, y = (1, 2, 3)
# Error: too many values to unpack
```

To avoid this, use extended unpacking:

```
x, *y = (1, 2, 3)  # x = 1, y = [2, 3]
```

DIFFERENCE BETWEEN SORT AND SORTED FUNCTION?

1. `sort()` Method

Features:

- `sort()` is a **list method** — it works **only on lists**.
- It **modifies the original list** (called **in-place sorting**).
- It does **not return a new list**, it returns `None`.

Syntax:

```
list.sort(reverse=False, key=None)
```

Example:

```
numbers = [4, 2, 1, 5, 3]
numbers.sort()
print(numbers)  # Output: [1, 2, 3, 4, 5]
```

Note:

```
result = numbers.sort()
print(result)  # Output: None
```

2. `sorted()` Function

Features:

- `sorted()` is a **built-in function**, not specific to lists.
- It works on **any iterable**: lists, tuples, strings, dictionaries, etc.
- It **returns a new sorted list**.
- The **original object remains unchanged**.

Syntax:

```
sorted(iterable, reverse=False, key=None)
```

Example:

```
numbers = [4, 2, 1, 5, 3]
new_list = sorted(numbers)
print(new_list)   # Output: [1, 2, 3, 4, 5]
print(numbers)    # Original list is unchanged
```

Comparison Table

Feature	`sort()`	`sorted()`
Type	List method	Built-in function
Works on	Only lists	Any iterable (list, tuple, string, etc.)
Returns	`None`	A new sorted list
Modifies original?	Yes (in-place)	No
Can be used with tuple/string?	✗ No	✓ Yes
Syntax	`list.sort()`	`sorted(iterable)`

Example with Tuple

```
t = (3, 1, 4)
print(sorted(t))  #  Output: [1, 3, 4]

# t.sort()  This will raise an error:
# AttributeError: 'tuple' object has no attribute 'sort'
```

Example with Reverse Sort

```
lst = [3, 1, 4]
lst.sort(reverse=True)
print(lst)  # Output: [4, 3, 1]

t = (3, 1, 4)
print(sorted(t, reverse=True))  # Output: [4, 3, 1]
```

Conditional Statements in Python: `if`, `if-else`, `elif`, and Nested `if`

Conditional statements are used to make decisions in Python programs. Based on certain conditions, different blocks of code are executed.

1. `if` Statement

Definition:
The `if` statement checks a condition. If the condition is **True**, the code inside the `if` block runs.

Syntax:

```
if condition:
    # code to execute if condition is True
```

Example:

```
age = 18
if age >= 18:
    print("You are eligible to vote.")
```

Output:

```
You are eligible to vote.
```

2. `if-else` Statement

Definition:
The `if-else` statement checks a condition.

- If it is **True**, the `if` block runs.
- If it is **False**, the `else` block runs.

Syntax:

```
if condition:
    # code if condition is True
else:
    # code if condition is False
```

Example:

```
age = 16
if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

Output:

```
You are not eligible to vote.
```

3. `elif` Statement (Short for "else if")

Definition:
Use `elif` when you have multiple conditions to check.

- Python checks the conditions one by one.
- The first condition that is `True` gets executed.
- If none are `True`, the `else` block runs (if present).

Syntax:

```
if condition1:
    # code if condition1 is True
elif condition2:
    # code if condition2 is True
elif condition3:
    # code if condition3 is True
else:
    # code if all conditions are False
```

Example:

```
marks = 75

if marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")
elif marks >= 60:
    print("Grade: C")
else:
    print("Grade: F")
```

Output:

```
Grade: B
```

4. Nested `if` Statement

Definition:
A nested `if` is an `if` statement inside another `if` or `else` block.
Used for **complex decisions** where multiple levels of checks are needed.

Syntax:

```
if condition1:
    if condition2:
        # code if both condition1 and condition2 are True
    else:
        # code if condition1 is True but condition2 is False
else:
    # code if condition1 is False
```

Example:

```
age = 20
citizen = "Indian"

if age >= 18:
    if citizen == "Indian":
        print("Eligible to vote in India.")
    else:
        print("Not eligible to vote in India.")
else:
    print("Too young to vote.")
```

Output:

```
Eligible to vote in India.
```

Logical Operators for Conditions

You can use **logical operators** to combine multiple conditions:

Operator	Meaning	Example
`And`	Both conditions must be True	`if a > 10 and b < 5:`
`Or`	At least one condition True	`if a > 10 or b < 5:`
`Not`	Reverses the condition	`if not is_logged_in:`

Example:

```
age = 25
has_id = True

if age >= 18 and has_id:
    print("Entry allowed.")
else:
    print("Entry denied.")
```

Output:

```
Entry allowed.
```

1. `if` Statement – 5 Examples

```
# Example 1
temperature = 30
if temperature > 25:
    print("It's a hot day.")

# Example 2
number = 10
if number % 2 == 0:
    print("Even number.")

# Example 3
name = "Alice"
if name == "Alice":
    print("Hello, Alice!")

# Example 4
marks = 85
if marks >= 80:
    print("You passed with distinction.")

# Example 5
is_raining = True
if is_raining:
    print("Take an umbrella.")
```

2. `if-else` Statement – 5 Examples

```
# Example 1
age = 17
if age >= 18:
    print("Adult")
else:
    print("Minor")

# Example 2
password = "1234"
if password == "admin":
    print("Access granted")
else:
    print("Access denied")

# Example 3
speed = 50
if speed > 60:
    print("Over Speed Limit")
else:
    print("Within Speed Limit")

# Example 4
number = -5
if number >= 0:
    print("Positive number")
else:
    print("Negative number")

# Example 5
score = 40
if score >= 50:
    print("Pass")
else:
    print("Fail")
```

3. `elif` Statement – 5 Examples

```
# Example 1
grade = 72
if grade >= 90:
    print("A")
elif grade >= 80:
    print("B")
elif grade >= 70:
    print("C")
else:
    print("Fail")

# Example 2
day = "Wednesday"
if day == "Monday":
    print("Start of week")
elif day == "Wednesday":
    print("Midweek")
elif day == "Friday":
    print("Weekend coming")
else:
    print("Just another day")

# Example 3
num = 0
if num > 0:
    print("Positive")
elif num < 0:
    print("Negative")
else:
    print("Zero")

# Example 4
size = "M"
if size == "S":
    print("Small")
elif size == "M":
    print("Medium")
elif size == "L":
    print("Large")
else:
    print("Invalid size")

# Example 5
marks = 45
if marks >= 90:
    print("Excellent")
elif marks >= 75:
    print("Very Good")
elif marks >= 60:
    print("Good")
elif marks >= 50:
    print("Average")
else:
    print("Poor")
```

4. Nested `if` Statement – 5 Examples

```
# Example 1
age = 20
country = "India"
if age >= 18:
    if country == "India":
```

```
        print("Eligible to vote")
    else:
        print("Not eligible to vote in India")
else:
    print("Too young to vote")

# Example 2
username = "admin"
password = "1234"
if username == "admin":
    if password == "1234":
        print("Login successful")
    else:
        print("Wrong password")
else:
    print("Unknown user")

# Example 3
number = 15
if number > 0:
    if number % 2 == 0:
        print("Positive even number")
    else:
        print("Positive odd number")
else:
    print("Not positive")

# Example 4
student = True
grade = 85
if student:
    if grade >= 80:
        print("Excellent student")
    else:
        print("Average student")
else:
    print("Not a student")

# Example 5
income = 50000
tax_paid = True
if income > 40000:
    if tax_paid:
        print("All good")
    else:
        print("Please pay taxes")
else:
    print("No tax needed")
```

String Data Type in Python

1. What is a String?

- A **string** is a **sequence of characters** enclosed within quotes.
- It can contain letters, numbers, symbols, spaces — anything that can be typed.

2. How to Create Strings

You can create strings using:

- **Single quotes** `'...'`
- **Double quotes** `"..."`
- **Triple quotes** `'''...'''` **or** `"""..."""` (for multi-line strings)

Examples:

```
str1 = 'Hello'
str2 = "World"
str3 = '''This is
a multi-line
string.'''
str4 = """Also
multi-line
string."""
```

3. String Characteristics

- Strings are **immutable** — once created, you **cannot change** individual characters.
- You can create a new string based on operations but can't modify the original string directly.

```
s = "Python"
# s[0] = 'p'  # ✗ This will raise an error because strings are immutable
```

4. Common String Operations

a. Concatenation

Joining two or more strings with the `+` operator.

```
greeting = "Hello"
name = "Alice"
message = greeting + " " + name
print(message)  # Output: Hello Alice
```

b. Repetition

Repeating strings using `*`.

```
print("ha" * 3)  # Output: hahaha
```

c. Accessing Characters (Indexing)

Strings are indexed starting from 0.

```
s = "Python"
print(s[0])  # Output: P
print(s[5])  # Output: n
```

Negative indexing counts from the end:

```
print(s[-1])  # Output: n (last character)
print(s[-2])  # Output: o (second last)
```

d. Slicing

Extract a substring from a string.

Syntax: `string[start:end]` where `start` is inclusive, `end` is exclusive.

```
s = "Python"
print(s[0:3])  # Output: Pyt (characters at index 0,1,2)
print(s[2:])   # Output: thon (from index 2 to end)
print(s[:4])   # Output: Pyth (from start to index 3)
print(s[-4:-1])# Output: tho (negative indices)
```

1. Assume a value for `s`:

Let's say:
`s = "Python"`

2. Understanding Negative Indexing:

In Python, strings can be accessed using:

- **Positive indexing** (from left to right)
- **Negative indexing** (from right to left)

Example string: `"Python"`

Character	P	y	t	H	o	n
Positive Index	0	1	2	3	4	5
Negative Index	-6	-5	-4	-3	-2	-1

3. What does `s[-4:-1]` mean?

- This is a **slice** operation: `s[start:end]`
- It **starts from index -4** (which is `'t'`)
- It **goes up to but does NOT include -1** (which is `'n'`)

So the characters at indices:

- `-4` → `'t'`
- `-3` → `'h'`
- `-2` → `'o'`
- `-1` → `'n'` (not included)

Hence, the slice is:

```
s[-4:-1] = 't' + 'h' + 'o' = "tho"
```

Final Output:

```
print(s[-4:-1])  # Output: tho
```

5. String Methods

Python has many built-in methods to work with strings.

Method	Description	Example
`lower()`	Converts string to lowercase	`"Hello".lower()` → `"hello"`
`upper()`	Converts string to uppercase	`"Hello".upper()` → `"HELLO"`
`strip()`	Removes whitespace from start and end	`" hello ".strip()` → `"hello"`
`replace(old, new)`	Replaces occurrences of substring	`"hello".replace("l", "p")` → `"heppo"`
`split(separator)`	Splits string into a list by separator	`"a,b,c".split(",")` → `['a', 'b', 'c']`
`join(iterable)`	Joins elements of an iterable into string	`",".join(['a','b','c'])` → `"a,b,c"`
`find(sub)`	Finds index of first occurrence of substring	`"hello".find("l")` → `2`
`count(sub)`	Counts occurrences of substring	`"hello".count("l")` → `2`
`startswith(prefix)`	Checks if string starts with prefix	`"hello".startswith("he")` → `True`
`endswith(suffix)`	Checks if string ends with suffix	`"hello".endswith("lo")` → `True`

Example:

```
text = "  Hello, Python!  "
print(text.lower())       # "  hello, python!  "
print(text.strip())       # "Hello, Python!"
print(text.replace("Python", "World"))  # "  Hello, World!  "
print(text.split(","))    # ['  Hello', ' Python!  ']
```

7. Raw Strings

Prefix string with `r` to treat backslashes literally (useful for Windows paths or regex).

```
path = r"C:\Users\Name"
print(path)  # Output: C:\Users\Name
```

8. String Formatting

Insert values into strings in readable ways.

a. Using `%` operator

```
name = "Alice"
age = 25
print("My name is %s and I am %d years old." % (name, age))
```

b. Using `str.format()`

```
print("My name is {} and I am {} years old.".format(name, age))
print("My name is {0} and I am {1} years old.".format(name, age))
print("My name is {name} and I am {age} years old.".format(name="Bob", age=30))
```

c. f-Strings (Python 3.6+)

```
print(f"My name is {name} and I am {age} years old.")
```

9. Immutability of Strings

Strings **cannot be changed** once created.

```
s = "Hello"
# s[0] = 'h'  # ✕ Error

# But you can create a new string:
s2 = 'h' + s[1:]
print(s2)  # "hello"
```

What is a Set in Python?

A **set** is a **built-in data type** in Python that is:

- **Unordered**: Elements have no index or order.
- **Unindexed**: You can't access items using indexes.
- **Mutable**: You can add or remove items.
- **No duplicates**: It automatically removes duplicate elements.

Syntax:

```
my_set = {1, 2, 3}
```

Or using the `set()` constructor:

```
my_set = set([1, 2, 3])
```

Note: An empty set must be created with `set()`, not `{}`. `{}` creates an empty dictionary.

Key Properties of Sets

Feature	Description
Unordered	No fixed order of elements
Unique items	Duplicate elements are automatically removed
Mutable	Can add or remove items
Iterable	Can loop through with `for` loop

Example 1: Creating a Set

```
my_set = {1, 2, 3, 4, 5}
print(my_set)
```

Output:

```
{1, 2, 3, 4, 5}
```

Example 2: Duplicate Removal

```
my_set = {1, 2, 2, 3, 3, 4}
print(my_set)
```

Output:

```
{1, 2, 3, 4}
```

Adding Elements:

`add()` – Add one item

```
s = {1, 2}
s.add(3)
print(s)  # {1, 2, 3}
```

`update()` – Add multiple items

```
s = {1, 2}
s.update([3, 4], {5, 6})
print(s)  # {1, 2, 3, 4, 5, 6}
```

Removing Elements

`remove()` – Removes item; raises error if not found

```
s = {1, 2, 3}
s.remove(2)
print(s)  # {1, 3}
```

`discard()` – Removes item; no error if not found

```
s = {1, 2, 3}
s.discard(4)  # No error
print(s)
```

`pop()` – Removes a random item

```
s = {1, 2, 3}
s.pop()
print(s)
```

`clear()` – Empties the set

```
s = {1, 2, 3}
s.clear()
print(s)  # Output: set()
```

Looping Through a Set

```
s = {10, 20, 30}
for item in s:
    print(item)
```

Set Operations (Like Math)

Let's assume:

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

1. Union: All unique elements from both

```
print(A | B)             # {1, 2, 3, 4, 5, 6}
print(A.union(B))
```

2. Intersection: Common elements

```
print(A & B)             # {3, 4}
print(A.intersection(B))
```

3. Difference: Elements in A but not in B

```
print(A - B)             # {1, 2}
print(A.difference(B))
```

4. Symmetric Difference: Elements in A or B, but not both

```
print(A ^ B)             # {1, 2, 5, 6}
print(A.symmetric_difference(B))
```

Membership Test

```
s = {1, 2, 3}
print(2 in s)     # True
print(5 not in s) # True
```

Things You Can't Do with Sets

- No indexing: `s[0]` → ✗ Error
- No slicing: `s[1:3]` → ✗ Error
- Cannot contain **mutable items** like lists or other sets as elements (but can contain tuples)

```
# ✗ Invalid
# s = {[1, 2], [3, 4]}  # Will raise error

# ✓ Valid
s = {(1, 2), (3, 4)}
```

When to Use Sets?

- When you need **unique values** (e.g., removing duplicates from a list).
- When you want to perform **mathematical set operations**.
- For **fast membership testing** (`in`/`not in`) — faster than lists.

Example: Remove duplicates from a list

```
numbers = [1, 2, 2, 3, 4, 4, 5]
unique = set(numbers)
print(unique)  # {1, 2, 3, 4, 5}
```

What is a Dictionary?

A dictionary is a mutable, unordered collection of key-value pairs. Keys must be unique and immutable (strings, numbers, tuples), while values can be any data type.

Key Characteristics

✓ **Key-Value Pairs:** Stores data as key: value pairs
✓ **Mutable:** Can add, remove, or modify items after creation
✓ **Unordered:** Items have no specific order (until Python 3.7+ where insertion order is preserved)
✓ **Keys must be unique and immutable**
✓ **Values can be any data type**

Creating Dictionaries

```
# Empty dictionary
empty_dict = {}
empty_dict2 = dict()
print(empty_dict)  # {}

# Dictionary with items
student = {
    "name": "Alice",
    "age": 20,
    "courses": ["Math", "Physics"]
}
print(student)  # {'name': 'Alice', 'age': 20, 'courses': ['Math', 'Physics']}

# Using dict() constructor
person = dict(name="Bob", age=25, city="New York")
print(person)  # {'name': 'Bob', 'age': 25, 'city': 'New York'}

# From list of tuples
fruits = dict([("apple", 5), ("banana", 3), ("orange", 7)])
print(fruits)  # {'apple': 5, 'banana': 3, 'orange': 7}
```

Accessing Values

```
student = {"name": "Alice", "age": 20, "courses": ["Math", "Physics"]}

# Using square brackets
print(student["name"])  # Alice

# Using get() method (safer)
print(student.get("age"))  # 20
print(student.get("grade"))  # None (key doesn't exist)
print(student.get("grade", "Not Available"))  # Not Available (default value)

# Get all keys, values, and items
print(student.keys())  # dict_keys(['name', 'age', 'courses'])
print(student.values())  # dict_values(['Alice', 20, ['Math', 'Physics']])
print(student.items())  # dict_items([('name', 'Alice'), ('age', 20), ('courses', ['Math',
'Physics'])])
```

Adding and Modifying Items

```
student = {"name": "Alice", "age": 20}

# Adding new key-value pair
student["grade"] = "A"
print(student)  # {'name': 'Alice', 'age': 20, 'grade': 'A'}

# Modifying existing value
student["age"] = 21
print(student)  # {'name': 'Alice', 'age': 21, 'grade': 'A'}

# Using update() to add multiple items
student.update({"age": 22, "city": "London", "phone": "123-4567"})
print(student)  # {'name': 'Alice', 'age': 22, 'grade': 'A', 'city': 'London', 'phone':
'123-4567'}
```

Removing Items

```
student = {"name": "Alice", "age": 20, "grade": "A", "city": "London"}

# Using del keyword
del student["city"]
print(student)  # {'name': 'Alice', 'age': 20, 'grade': 'A'}

# Using pop() method
age = student.pop("age")
print(age)  # 20
print(student)  # {'name': 'Alice', 'grade': 'A'}

# Using popitem() - removes last inserted item (Python 3.7+)
last_item = student.popitem()
print(last_item)  # ('grade', 'A')
print(student)  # {'name': 'Alice'}

# Clear all items
student.clear()
print(student)  # {}
```

Nested Dictionaries

```
# Nested dictionary example
school = {
    "student1": {
        "name": "Alice",
        "age": 20,
        "courses": ["Math", "Physics"]
    },
    "student2": {
        "name": "Bob",
        "age": 22,
        "courses": ["Chemistry", "Biology"]
    }
}

# Access nested values
print(school["student1"]["name"])  # Alice
print(school["student2"]["courses"][0])  # Chemistry

# Add new nested dictionary
school["student3"] = {"name": "Charlie", "age": 21}
print(school["student3"])
```

Useful Methods

```
student = {"name": "Alice", "age": 20, "grade": "A"}

# Check if key exists
print("name" in student)  # True
print("phone" in student)  # False

# Length of dictionary
print(len(student))  # 3

# Create copy
student_copy = student.copy()
print(student_copy)

# Set default value if key doesn't exist
phone = student.setdefault("phone", "000-0000")
```

```
print(phone)  # 000-0000
print(student)  # {'name': 'Alice', 'age': 20, 'grade': 'A', 'phone': '000-0000'}
```

Practical Examples

Example 1: Student Gradebook

```
# Gradebook system using dictionary
gradebook = {
    "Alice": {"Math": 85, "Science": 92, "English": 78},
    "Bob": {"Math": 88, "Science": 95, "English": 82},
    "Charlie": {"Math": 92, "Science": 90, "English": 85}
}

# Calculate average grade for each student
for student, grades in gradebook.items():
    average = sum(grades.values()) / len(grades)
    print(f"{student}: Average Grade = {average:.2f}")

# Add new student
gradebook["Diana"] = {"Math": 79, "Science": 85, "English": 88}

# Update a grade
gradebook["Alice"]["Math"] = 90
```

Example 2: Configuration Settings

```
# Application configuration using dictionary
config = {
    "database": {
        "host": "localhost",
        "port": 5432,
        "name": "mydb",
        "user": "admin",
        "password": "secret"
    },
    "server": {
        "host": "0.0.0.0",
        "port": 8000,
        "debug": True
    },
    "features": {
        "logging": True,
        "cache": False,
        "api_rate_limit": 1000
    }
}

# Access configuration values
db_host = config["database"]["host"]
server_port = config["server"]["port"]
print(f"Database: {db_host}, Server Port: {server_port}")

# Modify configuration
config["server"]["debug"] = False
config["features"]["cache"] = True
```

Example 3: Inventory Management

```
# Simple inventory system
inventory = {
    "001": {"name": "Laptop", "price": 999.99, "quantity": 15},
    "002": {"name": "Mouse", "price": 19.99, "quantity": 50},
    "003": {"name": "Keyboard", "price": 49.99, "quantity": 25}
```

```
}

# Add new product
inventory["004"] = {"name": "Monitor", "price": 199.99, "quantity": 10}

# Update stock
def update_stock(product_id, quantity_sold):
    if product_id in inventory:
        inventory[product_id]["quantity"] -= quantity_sold
        print(f"Updated stock for {inventory[product_id]['name']}")
    else:
        print("Product not found")

update_stock("001", 3)  # Sells 3 laptops
print(inventory["001"]["quantity"])  # 12

# Check low stock items
low_stock = {id: details for id, details in inventory.items() if details["quantity"] < 20}
print("Low stock items:", low_stock)
```

NOTE: Dictionaries are one of Python's most powerful data structures, perfect for storing and retrieving data by unique keys!

Python Looping

1. What is Looping?

Looping means **repeating a block of code** multiple times until a certain condition is met. It's very useful in programming when you need to perform repetitive tasks.

2. Types of Loops in Python

2.1 For Loop

We use the **for loop** when we want to iterate over a sequence (like list, string, tuple, range).

Syntax:

```
for variable in sequence:
    # code block
```

- `variable` takes each element of the sequence in every iteration.
- The indented block is the loop body, which executes repeatedly.

2.2 While Loop

The **while loop** is used when you want to repeat code **as long as a condition is true**.

Syntax:

```
while condition:
    # code block
```

- The loop runs while the condition is true.
- Once the condition becomes false, the loop stops.

3. For Loop in Detail

Example 1: Printing items of a list

```
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)
```

Output:

```
apple
banana
cherry
```

Example 2: Printing characters of a string

```
for letter in "Python":
    print(letter)
```

Output:

```
P
y
t
h
o
n
```

Example 3: Using Range

```
for i in range(1, 6):
    print(i)
```

Output:

```
1
2
3
4
5
```

About `range()`:

- `range(stop)` generates numbers from 0 up to stop-1.
- `range(start, stop)` generates numbers from start up to stop-1.
- `range(start, stop, step)` generates numbers with a step size.

Example:

```
for i in range(0, 10, 2):
    print(i)
```

Output:

```
0
2
4
6
8
```

4. While Loop in Detail

Example 1: Basic while loop

```
count = 0
while count < 5:
    print(count)
    count += 1  # count = count + 1
```

Output:
```
0
1
2
3
4
```

Example 2: Loop based on user input

```
user_input = ""
while user_input.lower() != "exit":
    user_input = input("Type 'exit' to stop: ")
print("Loop stopped.")
```

5. Loop Control Statements

5.1 `break`

Stops the loop immediately.

Example:

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

Output:
```
0
1
2
3
4
```

5.2 `continue`

Skips the current iteration and moves to the next.

Example:
```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

Output:
```
0
1
3
4
```

5.3 `else` with loops

The `else` block runs if the loop completes normally (without a `break`).

Example:

```
for i in range(3):
    print(i)
else:
    print("Loop completed without break")
```

Example with break:

```
for i in range(5):
    if i == 3:
        break
    print(i)
else:
    print("This will NOT print because loop broke early")
```

6. Nested Loops (Loops inside loops)

Nested loops mean running a loop inside another loop.

Example:

```
for i in range(1, 4):
    for j in range(1, 3):
        print(f"i = {i}, j = {j}")
```

Output:

```
i = 1, j = 1
i = 1, j = 2
i = 2, j = 1
i = 2, j = 2
i = 3, j = 1
i = 3, j = 2
```

7. Practical Examples

7.1 Sum of First N Numbers

```
n = 10
total = 0
for i in range(1, n+1):
    total += i
print("Sum:", total)
```

7.2 Multiplication Table of a Number

```
num = 7
for i in range(1, 11):
    print(f"{num} x {i} = {num*i}")
```

8. Infinite Loops

- If the condition of a `while` loop never becomes false, the loop runs forever.
- To avoid infinite loops, ensure the condition will eventually become false.

Example of an infinite loop:

```
while True:
    print("This will print forever!")
```

Loops with List, Tuple, Set, Dictionary

1. With List

For Loop Examples:

```
# Example 1: Print all elements
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)

# Example 2: Print index and element
for i in range(len(fruits)):
    print(i, fruits[i])

# Example 3: Sum of numbers in a list
numbers = [2, 4, 6, 8]
total = 0
for num in numbers:
    total += num
print("Sum:", total)

# Example 4: Print elements in uppercase
for fruit in fruits:
    print(fruit.upper())

# Example 5: Break the loop if 'banana' is found
for fruit in fruits:
    if fruit == 'banana':
        print("Banana found!")
        break
```

While Loop Examples:

```
# Example 1: Print all elements using while loop
i = 0
while i < len(fruits):
    print(fruits[i])
    i += 1

# Example 2: Sum of numbers using while loop
numbers = [1, 3, 5, 7]
total = 0
i = 0
while i < len(numbers):
    total += numbers[i]
    i += 1
print("Sum:", total)
```

```
# Example 3: Print elements until 'cherry' is found
i = 0
while i < len(fruits):
    if fruits[i] == 'cherry':
        print("Cherry found!")
        break
    print(fruits[i])
    i += 1

# Example 4: Print elements with their index
i = 0
while i < len(fruits):
    print(f"Index {i}: {fruits[i]}")
    i += 1

# Example 5: Print elements in reverse order
i = len(fruits) - 1
while i >= 0:
    print(fruits[i])
    i -= 1
```

2. With Tuple

For Loop Examples:

```
# Example 1: Print all tuple elements
colors = ('red', 'green', 'blue')
for color in colors:
    print(color)

# Example 2: Print index and value
for i in range(len(colors)):
    print(i, colors[i])

# Example 3: Check if an element exists
for color in colors:
    if color == 'green':
        print("Green is in the tuple")

# Example 4: Print elements in uppercase
for color in colors:
    print(color.upper())

# Example 5: Count vowels in each element
for color in colors:
    count = 0
    for ch in color:
        if ch in 'aeiou':
            count += 1
    print(f"{color} has {count} vowels")
```

While Loop Examples:

```
# Example 1: Print all elements using while loop
i = 0
while i < len(colors):
    print(colors[i])
    i += 1

# Example 2: Print elements until 'green' is found
i = 0
while i < len(colors):
    if colors[i] == 'green':
        print("Found green!")
```

```
        break
    print(colors[i])
    i += 1

# Example 3: Print elements with index
i = 0
while i < len(colors):
    print(f"Index {i}: {colors[i]}")
    i += 1

# Example 4: Print length of each element
i = 0
while i < len(colors):
    print(f"{colors[i]} length is {len(colors[i])}")
    i += 1

# Example 5: Print elements in reverse order
i = len(colors) - 1
while i >= 0:
    print(colors[i])
    i -= 1
```

3. With Set

Note: Sets are unordered, so they don't support indexing directly.

For Loop Examples:

```
# Example 1: Print all elements in the set
animals = {'cat', 'dog', 'rabbit'}
for animal in animals:
    print(animal)

# Example 2: Check if 'dog' is in the set and print a message
for animal in animals:
    if animal == 'dog':
        print("Dog is here!")

# Example 3: Print elements in uppercase
for animal in animals:
    print(animal.upper())

# Example 4: Count characters in each element
for animal in animals:
    print(f"{animal} has {len(animal)} letters")

# Example 5: Print elements with a star prefix
for animal in animals:
    print(f"* {animal}")
```

While Loop Examples:

Since sets don't support indexing, convert to a list first.

```
# Example 1: Print all elements using while loop (converted to list)
animals = {'cat', 'dog', 'rabbit'}
animal_list = list(animals)
i = 0
while i < len(animal_list):
    print(animal_list[i])
    i += 1
```

```
# Example 2: Print elements until 'dog' is found
i = 0
while i < len(animal_list):
    if animal_list[i] == 'dog':
        print("Dog found!")
        break
    print(animal_list[i])
    i += 1

# Example 3: Print elements in uppercase
i = 0
while i < len(animal_list):
    print(animal_list[i].upper())
    i += 1

# Example 4: Print elements with length
i = 0
while i < len(animal_list):
    print(f"{animal_list[i]} length is {len(animal_list[i])}")
    i += 1

# Example 5: Print elements in reverse order
i = len(animal_list) - 1
while i >= 0:
    print(animal_list[i])
    i -= 1
```

4. With Dictionary

For Loop Examples:

```
person = {'name': 'Alice', 'age': 25, 'city': 'New York'}

# Example 1: Print all keys
for key in person:
    print(key)

# Example 2: Print all values
for value in person.values():
    print(value)

# Example 3: Print key-value pairs
for key, value in person.items():
    print(f"{key} -> {value}")

# Example 4: Print only keys that start with 'c'
for key in person:
    if key.startswith('c'):
        print(key)

# Example 5: Increase age by 1
for key in person:
    if key == 'age':
        person[key] += 1
print(person)
```

While Loop Examples:

Convert keys, values, or items to a list for indexing.

```
person = {'name': 'Alice', 'age': 25, 'city': 'New York'}
keys = list(person.keys())
```

```
# Example 1: Print all keys using while loop
i = 0
while i < len(keys):
    print(keys[i])
    i += 1

# Example 2: Print all values using while loop
values = list(person.values())
i = 0
while i < len(values):
    print(values[i])
    i += 1

# Example 3: Print key-value pairs using while loop
items = list(person.items())
i = 0
while i < len(items):
    print(f"{items[i][0]} -> {items[i][1]}")
    i += 1

# Example 4: Print keys containing 'a'
i = 0
while i < len(keys):
    if 'a' in keys[i]:
        print(keys[i])
    i += 1

# Example 5: Increase age by 5 inside while loop
i = 0
while i < len(keys):
    if keys[i] == 'age':
        person[keys[i]] += 5
    i += 1
print(person)
```

What is a frozenset?

A frozenset is an immutable, unordered collection of unique elements. Unlike regular sets, frozensets cannot be modified after creation.

Key Characteristics

✅ **Immutable:** Cannot add, remove, or modify elements after creation
✅ **Unordered:** Elements have no specific order
✅ **Hashable:** Can be used as keys in dictionaries or elements in other sets
✅ **Contains unique elements:** No duplicates allowed

Creating frozensets

```
# Empty frozenset
empty_fs = frozenset()
print(empty_fs)  # frozenset()

# From iterable
fs1 = frozenset([1, 2, 3, 4, 5])
fs2 = frozenset("hello")  # frozenset({'h', 'e', 'l', 'o'})
fs3 = frozenset({1, 2, 3, 2, 1})  # frozenset({1, 2, 3}) - duplicates removed

print(fs1)  # frozenset({1, 2, 3, 4, 5})
```

```
print(fs2)  # frozenset({'h', 'e', 'l', 'o'})
```

Comparison Operations

```
fs1 = frozenset([1, 2, 3])
fs2 = frozenset([1, 2])
fs3 = frozenset([1, 2, 3])

# Subset
print(fs2 <= fs1)  # True (subset)
print(fs2 < fs1)   # True (proper subset)

# Superset
print(fs1 >= fs2)  # True (superset)
print(fs1 > fs2)   # True (proper superset)

# Equality
print(fs1 == fs3)  # True
print(fs1 == fs2)  # False
```

Useful Methods

```
fs = frozenset([1, 2, 3, 4, 5])

# Length
print(len(fs))  # 5

# Minimum and maximum
print(min(fs))  # 1
print(max(fs))  # 5

# Sum
print(sum(fs))  # 15
```

Conversion:

```
# Convert to other types
fs = frozenset([1, 2, 3, 4])

# To list
lst = list(fs)
print(lst)  # [1, 2, 3, 4] (order may vary)

# To set
regular_set = set(fs)
print(regular_set)  # {1, 2, 3, 4}

# To tuple
tup = tuple(fs)
print(tup)  # (1, 2, 3, 4) (order may vary)
```

Limitations:

```
fs = frozenset([1, 2, 3])

# These operations will raise AttributeError
    fs.add(4)        # ✗ Cannot add
    fs.remove(1)     # ✗ Cannot remove
    fs.discard(2)    # ✗ Cannot discard
    fs.clear()       # ✗ Cannot clear
    fs.update([4])   # ✗ Cannot update
```

Best Practices

- Use frozensets when you need immutable, hashable collections
- Perfect for dictionary keys or set elements
- Use when data shouldn't be modified after creation
- More memory efficient than tuples for membership testing

Comprehension:

In Python, **comprehensions** provide a concise and readable way to create **lists**, **dictionaries**, **sets**, or **generators** using a single line of code. They are widely used for transforming or filtering data from iterables like lists, tuples, sets, etc.

Python supports the following types of comprehensions:

1. List Comprehension

Purpose:

To create a **new list** by applying an expression to each item in an iterable.

Syntax:

```
[expression for item in iterable if condition]
```

Examples:

1. Basic list comprehension

```
squares = [x**2 for x in range(5)]
print(squares)  # Output: [0, 1, 4, 9, 16]
```

Step-by-Step Explanation:

Step 1: `range(5)`

- `range(5)` is a built-in Python function that generates a sequence of numbers starting from `0` up to, but not including, `5`.

```
list(range(5))  # Output: [0, 1, 2, 3, 4]
```

So `x` will take each value from this sequence: `0, 1, 2, 3, 4`.

Step 2: `x**2`

- This is the expression that is **applied to each item `x`** from the range.
- `x**2` means "**x squared**" (i.e., `x` raised to the power of 2).

Let's calculate:

- `0**2 = 0`

- `1**2 = 1`
- `2**2 = 4`
- `3**2 = 9`
- `4**2 = 16`

Step 3: List Comprehension

The list comprehension `[x**2 for x in range(5)]` can be **read like this** in plain English:

"For each number `x` in the range from 0 to 4, compute `x**2`, and collect all results into a list."

Internally, Python builds this list by looping through `range(5)` and applying `x**2` to each item.

So this comprehension becomes:

```
squares = [0**2, 1**2, 2**2, 3**2, 4**2]
squares = [0, 1, 4, 9, 16]
```

Step 4: `print(squares)`

This line simply outputs the final list created in the previous step.

```
print(squares)
# Output: [0, 1, 4, 9, 16]
```

Equivalent Code Without List Comprehension (Using a `for` loop)

If we were to write this code **without list comprehension**, it would look like this:

```
squares = []
for x in range(5):
    squares.append(x**2)

print(squares)
# Output: [0, 1, 4, 9, 16]
```

As you can see, list comprehensions make your code **more concise and readable**, especially for simple transformations like this

2. With condition (filtering)

```
even_numbers = [x for x in range(10) if x % 2 == 0]
print(even_numbers)  # Output: [0, 2, 4, 6, 8]
```

Step-by-Step Explanation:

Step 1: `range(10)`

- `range(10)` generates numbers starting from `0` up to **but not including** `10`.
- That means:
- `list(range(10))  # ➜ [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`

So `x` will iterate over each of these numbers one by one.

Step 2: `x % 2 == 0` → The Condition

- The `%` **operator** is the modulo operator. It gives the **remainder** after dividing two numbers.
- `x % 2 == 0` means:

  "Is `x` divisible by 2 with no remainder?"

This is how you **check if a number is even**.

Let's check each number in `range(10)`:

`x`	`x % 2`	Is Even?
0	0	✓ Yes
1	1	✗ No
2	0	✓ Yes
3	1	✗ No
4	0	✓ Yes
5	1	✗ No
6	0	✓ Yes
7	1	✗ No
8	0	✓ Yes
9	1	✗ No

So, only these values satisfy the condition:
`0, 2, 4, 6, 8`

Step 3: `[x for x in range(10) if x % 2 == 0]`

This is a **list comprehension**.

You can read it as:

"For each number `x` in the range 0 to 9, include `x` in the list **only if** `x` is divisible by 2 (i.e., even)."

So it builds this list:

```
even_numbers = [0, 2, 4, 6, 8]
```

Step 4: `print(even_numbers)`

This line outputs the result of the list comprehension.

```
print(even_numbers)
# Output: [0, 2, 4, 6, 8]
```

Equivalent Code Without List Comprehension (Using `for` loop)

If we rewrote this without a list comprehension, it would look like this:

```
even_numbers = []
for x in range(10):
    if x % 2 == 0:
        even_numbers.append(x)

print(even_numbers)
# Output: [0, 2, 4, 6, 8]
```

NOTE: Same result, but more lines of code.

2. Dictionary Comprehension

Purpose:

To create a **new dictionary** from an iterable.

Syntax:

```
{key_expression: value_expression for item in iterable if condition}
```

Why Use Dictionary Comprehension?

- It's **faster**, more **compact**, and more **readable** than traditional loops.
- Very useful for **filtering**, **mapping**, or **transforming** data into key-value pairs.

Examples:

1. Basic dictionary comprehension

```
squares = {x: x**2 for x in range(5)}
print(squares)  # Output: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Step-by-Step Breakdown:

Step 1: `range(5)`

- `range(5)` generates a sequence of numbers from **0 to 4** (5 is **not included**).

```
list(range(5))  # ➜ [0, 1, 2, 3, 4]
```

So `x` will take each of the values:

```
0, 1, 2, 3, 4
```

Step 2: `{x: x**2 for x in range(5)}` → Dictionary Comprehension

This is a **dictionary comprehension**, which creates a dictionary in a compact way.

General syntax:

```
{key_expr: value_expr for item in iterable}
```

In this case:

- `key_expr = x`
- `value_expr = x**2`
- `iterable = range(5)`

So for every number `x` in `range(5)`, it creates a key-value pair where:

- **Key = `x`**
- **Value = `x**2` (i.e., square of x)**

Step 3: Iteration Process

Let's go through it manually:

Iteration	x	x**2	Dictionary Entry
1	0	0	`0: 0`
2	1	1	`1: 1`
3	2	4	`2: 4`
4	3	9	`3: 9`
5	4	16	`4: 16`

So, the resulting dictionary is:
`{0: 0, 1: 1, 2: 4, 3: 9, 4: 16}`

Step 4: `print(squares)`

This prints the final dictionary.

```
print(squares)
# Output: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

This shows each number from 0 to 4 as a **key**, and its **square as the value**.

Equivalent Code Without Dictionary Comprehension (Using `for` loop)

Here's how you'd write the same logic with a standard `for` loop:

```
squares = {}
for x in range(5):
    squares[x] = x**2

print(squares)
# Output: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

The dictionary comprehension is just a shorter and cleaner way to write this.

2. Filtering with condition

```
even_squares = {x: x**2 for x in range(10) if x % 2 == 0}
print(even_squares)  # Output: {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

Step-by-Step Explanation:

Step 1: `range(10)`

- `range(10)` generates numbers from **0 to 9** (10 is **not included**):

```
list(range(10))  # ➜ [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

So `x` will take each value in this list one by one.

Step 2: `if x % 2 == 0` — The Condition

This part **filters** the values:

- `%` is the **modulus operator** — it gives the **remainder**.
- `x % 2 == 0` checks if `x` is divisible by 2 (i.e., **even**).

So this condition will only allow **even numbers** through:

```
[0, 2, 4, 6, 8] ✓ pass the condition
[1, 3, 5, 7, 9] ✗ are skipped
```

Step 3: `{x: x**2 for x in range(10) if x % 2 == 0}`

This is a **dictionary comprehension**.

It builds key-value pairs where:

- **Key** = `x`
- **Value** = `x**2` (square of x)
- But only for those `x` that are **even**

Let's walk through each even number:

x	x % 2 == 0	x**2	Dictionary Entry
0	✓	0	`0: 0`
2	✓	4	`2: 4`
4	✓	16	`4: 16`
6	✓	36	`6: 36`
8	✓	64	`8: 64`

So the resulting dictionary is:

```
{0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

Step 4: `print(even_squares)`

This prints the dictionary created by the comprehension.

```
print(even_squares)
# Output: {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

Equivalent Code Without Dictionary Comprehension (Using a `for` loop)

Here's how you'd write the same logic using a traditional `for` loop:

```
even_squares = {}
```

```
for x in range(10):
    if x % 2 == 0:
        even_squares[x] = x**2

print(even_squares)
# Output: {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

NOTE: Same logic, but takes more lines of code.

3. From a list of tuples

```
data = [('a', 1), ('b', 2), ('c', 3)]
dict_data = {k: v for k, v in data}
print(dict_data)  # Output: {'a': 1, 'b': 2, 'c': 3}
```

🔍 Step-by-Step Explanation

Step 1: `data = [('a', 1), ('b', 2), ('c', 3)]`

- This is a list of **tuples**.
- Each tuple contains **two elements**: a **key** (string) and a **value** (integer).

```
data = [
    ('a', 1),
    ('b', 2),
    ('c', 3)
]
```

You can think of this list as representing key-value pairs that you'd find in a dictionary.

Step 2: `for k, v in data`

This is the loop part of the dictionary comprehension.

- It loops through each tuple in the `data` list.
- Each tuple is **unpacked** into:
  - `k`: the first element (will become the dictionary key)
  - `v`: the second element (will become the dictionary value)

Let's break it down iteration by iteration:

Iteration	Tuple	k	v
1	`('a', 1)`	'a'	1
2	`('b', 2)`	'b'	2
3	`('c', 3)`	'c'	3

Step 3: `{k: v for k, v in data}`

This is a **dictionary comprehension**, meaning it builds a dictionary dynamically.

- For each `(k, v)` pair in the list `data`, it creates an entry in the dictionary:
  - `key = k`
  - `value = v`

So it produces:

```
{
    'a': 1,
    'b': 2,
    'c': 3
}
```

Step 4: `print(dict_data)`

This line prints the dictionary created in the previous step.

```
print(dict_data)
# Output: {'a': 1, 'b': 2, 'c': 3}
```

Equivalent Code Without Dictionary Comprehension

Here's how you could write the same thing using a `for` loop:

```
data = [('a', 1), ('b', 2), ('c', 3)]
dict_data = {}
for k, v in data:
    dict_data[k] = v

print(dict_data)
# Output: {'a': 1, 'b': 2, 'c': 3}
```

3. Set Comprehension

Purpose:

To create a **new set**, which is an unordered collection of unique items.

Syntax:

```
{expression for item in iterable if condition}
```

Why Use Set Comprehension?

- Quick way to build a set from an iterable
- Automatically removes duplicates
- Cleaner and more Pythonic than a loop

Examples:

1. Basic set comprehension

```
squares = {x**2 for x in range(5)}
print(squares)  # Output: {0, 1, 4, 9, 16}
```

Step-by-Step Explanation:

This is an example of a **set comprehension** — a very compact way to create a set in Python.

Step 1: `range(5)`

- `range(5)` generates a sequence of numbers starting from **0 up to (but not including)** 5.

```
list(range(5))  # ➜ [0, 1, 2, 3, 4]
```

So, the variable `x` will take on the values:
`0, 1, 2, 3, 4`

Step 2: `x**2`

- This is the **expression being applied** to each value of `x`.
- `x**2` means "**x squared**" — in other words, raise `x` to the power of 2.

Let's compute the square of each number from the range:

x	x**2
0	0
1	1
2	4
3	9
4	16

Step 3: `{x**2 for x in range(5)}` → Set Comprehension

This syntax creates a **set**.

What is a set?

- A **set** is an **unordered**, **mutable**, and **unindexed** collection of **unique** elements.
- It removes **duplicates automatically** (though in this case, there are no duplicates).

So this line:
```
squares = {x**2 for x in range(5)}
```

is equivalent to:
```
squares = {0, 1, 4, 9, 16}
```

Step 4: `print(squares)`

This line prints the set:

```
print(squares)
# Output: {0, 1, 4, 9, 16}
```

Note: The order of elements in a set is **not guaranteed**, so you might see the elements printed in a different order like `{16, 0, 1, 4, 9}` — and that's completely normal for sets in Python.

Equivalent Code Without Set Comprehension

You can write the same logic using a `for` loop like this:

```
squares = set()
for x in range(5):
    squares.add(x**2)

print(squares)
```

```
# Output: {0, 1, 4, 9, 16}
```

2. Filtering

```
even_squares = {x**2 for x in range(10) if x % 2 == 0}
print(even_squares)  # Output: {0, 64, 4, 36, 16}
```

Python Functions:

1. What is a Function?

A **function** is a named block of code designed to perform a particular task. It allows you to encapsulate logic in one place and reuse it anywhere in your program.

Functions promote **modularity**, **code reuse**, and **organization**.

2. Defining and Calling Functions

Syntax:

```
def function_name(parameters):
    """
    Optional docstring describing the function.
    """
    # function body (statements)
    return result  # optional
```

- `def` keyword introduces the function.
- `function_name` follows naming rules (letters, digits, underscore, no spaces, can't start with digits).
- `parameters` (also called arguments or formal parameters) are optional.
- `return` is optional but used to send data back to the caller.

Example:

```
def greet():
    print("Hello!")
```

Call it like this:

```
greet()  # Output: Hello!
```

3. Function Parameters & Arguments

a) Positional Parameters

When you define parameters in a function, you expect arguments to be passed in order.

```
def add(a, b):
    return a + b

print(add(5, 3))  # 8
```

Here, `5` is assigned to `a`, `3` to `b`.

b) Keyword Arguments

You can explicitly specify which argument corresponds to which parameter, regardless of order.

```
print(add(b=4, a=10))  # 14
```

c) Default Parameters

You can provide default values to parameters so that if an argument is omitted, the default is used.

```
def greet(name="Guest"):
    print(f"Hello, {name}!")

greet()           # Hello, Guest!
greet("Alice")    # Hello, Alice!
```

Note: Parameters with default values must come after those without defaults.

4. Variable-Length Arguments

a) *args (Non-keyword Variable Arguments)

Use when you don’t know how many positional arguments will be passed.

```
def multiply(*args):
    product = 1
    for num in args:
        product *= num
    return product

print(multiply(2, 3))         # 6
print(multiply(2, 3, 4, 5))   # 120
```

Inside the function, `args` is a tuple of all passed positional arguments.

b) **kwargs (Keyword Variable Arguments)

Used when you want to accept any number of keyword arguments.

```
def print_person(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_person(name="Bob", age=30, city="New York")
```

Output:

```
name: Bob
age: 30
city: New York
```

Inside the function, `kwargs` is a dictionary of all keyword arguments.

5. Return Statement

Functions can send back data to the caller using `return`.

- If no return statement, function returns `None` by default.
- You can return any data type: numbers, strings, lists, dictionaries, tuples, objects, etc.
- You can return multiple values separated by commas — Python packs them into a tuple.

Example returning multiple values:

```
def get_min_max(numbers):
    return min(numbers), max(numbers)

min_val, max_val = get_min_max([2, 7, 1, 9])
print(min_val, max_val)  # 1 9
```

6. Function Annotations (Type Hints)

Python allows optional type hints to specify expected argument types and return types. These are for readability and tools; they don't enforce type checking at runtime.

```
def greet(name: str) -> None:
    print(f"Hello, {name}!")

def add(a: int, b: int) -> int:
    return a + b
```

Explanation:

- `def greet(name: str) -> None:`
  This line defines a function named `greet` that takes one parameter called `name`.
  - `name: str` is a **type hint** indicating that `name` is expected to be a string.
  - `-> None` is a **return type hint**, meaning this function does **not return any value** (or returns `None`).
- `print(f"Hello, {name}!")`
  This line prints a greeting message to the console using an **f-string** (formatted string literal). It inserts the value of `name` into the string.
  For example, if `name` is `"Alice"`, it will print:

  ```
  Hello, Alice!
  ```

7. Anonymous Functions (Lambda)

Lambda functions are small, anonymous functions defined using the `lambda` keyword. They can only contain expressions, not statements.

Syntax:

```
lambda arguments: expression
```

Example:

```
square = lambda x: x * x
print(square(7))  # 49
```

Lambdas are often used as arguments to other functions, like `map()`, `filter()`, and `sorted()`.

```
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x**2, numbers))
print(squares)  # [1, 4, 9, 16, 25]
```

Scope of Variables in Python (LEGB Rule)

Scope means the region of a program where a variable is accessible.
In Python, variable resolution follows the **LEGB Rule**:
Local → Enclosed → Global → Built-in

1. Local Scope

- A variable declared inside a function is **local** to that function.
- It cannot be accessed outside the function.

```
def func():
    x = 10   # Local variable
    print("Inside:", x)

func()
# print(x)  ❌ Error: x is not accessible outside
```

2. Enclosed Scope

- When a function is inside another function,
- The inner function can access variables of the **outer (enclosing) function**.

```
def outer():
    y = 20   # Enclosed scope
    def inner():
        print("Inner:", y)   # ✅ Accessible
    inner()

outer()
```

3. Global Scope

- A variable declared outside all functions is **global**.
- It can be accessed anywhere in the program.

```
z = 30   # Global variable

def func():
    print("Inside function:", z)   # ✅ Accessible

func()
print("Outside function:", z)      # ✅ Accessible
```

4. Built-in Scope

- These are predefined names in Python like `len()`, `print()`, `range()`, etc.
- Always available.

```
print(len([1,2,3]))  # ✅ len is built-in
```

Local vs Global Conflict

- If a local and global variable have the same name, **local variable gets priority**.

```
x = 100   # Global

def func():
    x = 50   # Local
    print("Inside:", x)

func()
print("Outside:", x)
# Output:
# Inside: 50
# Outside: 100
```

`global` Keyword

- Used to modify a **global variable inside a function**.

```
count = 0

def update():
    global count
    count += 1   # modifies global variable

update()
print(count)   # ✅ Output: 1
```

Interview-Ready Answer:

"In Python, variable scope defines where a variable is accessible. Python follows the **LEGB rule**:

- **Local**: Inside the current function.
- **Enclosed**: Inside outer (enclosing) functions.
- **Global**: Declared outside all functions, accessible everywhere.
- **Built-in**: Predefined Python functions and constants.

If there's a naming conflict, Python checks variables in the order **Local → Enclosed → Global → Built-in**. To modify global variables inside a function, we use the `global` keyword."

1. What is a Lambda (Anonymous) Function?

A **lambda function** is a **small, one-line anonymous function** in Python.

- "Anonymous" means **it has no name** (unlike normal functions created using `def`).
- It can take **any number of arguments** but can only have **one expression**.

Syntax:

```
lambda arguments: expression
```

The `expression` is automatically returned (no need to use `return` keyword).

2. Example — Basic Lambda Function

Example 1:

```
# Normal function
def add(x, y):
    return x + y

# Same function using lambda
add = lambda x, y: x + y

print(add(5, 3))   # Output: 8
```

NOTE: Both functions do the same thing, but the lambda version is shorter.

3. Why use Lambda Functions?

They are mostly used when:

- You need a **short function for a short time** (temporary use).
- You don't want to define a full function using `def`.
- You want to use it **inside another function** like `map()`, `filter()`, or `sorted()`.

4. Difference Between `def` and `lambda`

Feature	`def` Function	`lambda` Function
Name	Has a name	Anonymous (no name)
Number of lines	Can have multiple lines	Only one expression
Use of `return`	Must use `return`	Automatically returns the expression
Purpose	For reusable, complex code	For short, temporary logic
Example	`def add(x, y): return x+y`	`lambda x, y: x+y`

5. Limitations of Lambda Functions

❌ Can contain only **one expression**
❌ Cannot have statements like `if-else` (except inline form), `for`, `while`, `try`, etc.
❌ Cannot have docstrings or annotations
❌ Harder to debug or reuse (no function name)

6. When to Use Lambda Functions

Use lambda when:

- Function is **short and simple**
- Used **only once**
- Example: inside `map()`, `filter()`, `sorted()`, `reduce()`

Avoid lambda when:

- The logic is **complex**
- Function will be **reused or tested later**

7. Practice Examples

Example 1:

```
cube = lambda x: x ** 3
print(cube(4))   # Output: 64
```

Example 2:

```
check_even = lambda x: "Even" if x % 2 == 0 else "Odd"
print(check_even(5))  # Output: Odd
```

Example 3:

```
names = ["alex", "John", "maria", "sam"]

# Sort by length
print(sorted(names, key=lambda name: len(name)))

# Output: ['sam', 'alex', 'John', 'maria']
```

What is a Higher-Order Function?

A **Higher-Order Function (HOF)** is a **function that does any one (or both)** of the following:

1. **Takes another function as an argument**
2. **Returns a function as a result**

In short:

A function that works with another function is called a Higher-Order Function.

Example 1: Function passed as an argument

Let's see a simple example

```
def greet(name):
    return f"Hello, {name}!"

def display_message(func):
    # 'func' is another function passed as argument
    print(func("JOHN"))

display_message(greet)
```

Output:

```
Hello, JOHN!
```

Here,

- `greet` is a normal function.

- `display_message()` is a **higher-order function** because it **accepts another function** (`greet`) as its argument.

Example 2: Function returning another function

```
def outer():
    def inner():
        return "I am inside inner function"
    return inner   # returning a function, not calling it

my_func = outer()  # outer() returns inner function
print(my_func())   # call inner function
```

Output:

```
I am inside inner function
```

Note: `outer()` is a **higher-order function** because it **returns another function** (`inner`).

Why use Higher-Order Functions?

Reasons:

1. To make code **more reusable and modular**.
2. To **add behavior dynamically** to other functions (used in decorators).
3. To **process collections** easily using built-in HOFs like `map()`, `filter()`, `reduce()`.

Built-in Higher-Order Functions in Python

Python already provides many HOFs — the most common are:

Function	Description
`map(function, iterable)`	Applies a function to every element of an iterable
`filter(function, iterable)`	Selects elements that satisfy a condition
`reduce(function, iterable)`	Repeatedly applies a function to combine elements (from `functools`)
`sorted(iterable, key=function)`	Sorts data using a key function

Let's explore each one

1. `map()` — Apply a function to all elements

Syntax:

```
map(function, iterable)
```

Example:

```
numbers = [1, 2, 3, 4]
squares = list(map(lambda x: x**2, numbers))
print(squares)
```

Output:

```
[1, 4, 9, 16]
```

Note: Here, `map()` is a **higher-order function** because it takes another function (`lambda x: x**2`) as argument.

2. `filter()` — Filter items based on a condition

Syntax:

```
filter(function, iterable)
```

Example:

```
numbers = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)
```

Output:

```
[2, 4, 6]
```

Note: `filter()` keeps only those elements where the lambda returns `True`.

3. `reduce()` — Combine all items (from `functools`)

Syntax:

```
from functools import reduce
reduce(function, iterable)
```

Example:

```
from functools import reduce

numbers = [1, 2, 3, 4]
sum_all = reduce(lambda x, y: x + y, numbers)
print(sum_all)
```

Output:

```
10
```

Note: `reduce()` repeatedly applies the function:
(((1 + 2) + 3) + 4) = 10

4. `sorted()` — Custom sorting using a function

```
names = ["john", "Alice", "Bob", "alex"]
sorted_names = sorted(names, key=lambda x: x.lower())
print(sorted_names)
```

Output:

```
['alex', 'Alice', 'Bob', 'john']
```

Note: Here `key=lambda x: x.lower()` tells `sorted()` **how** to compare each element — again, a higher-order behavior.

Real-Life Example

Let's combine `map`, `filter`, and `reduce`

```
from functools import reduce

numbers = [1, 2, 3, 4, 5, 6]

# Step 1: double each number
doubled = map(lambda x: x * 2, numbers)

# Step 2: keep only even numbers
filtered = filter(lambda x: x % 2 == 0, doubled)

# Step 3: sum them up
result = reduce(lambda x, y: x + y, filtered)

print(result)
```

Output:

```
36
```

Step-by-step:

- Doubled: [2, 4, 6, 8, 10, 12]
- Even: [2, 4, 6, 8, 10, 12] (all are even)
- Reduced sum: 42

Generator Comprehension (Generator Expression)

Purpose:

To create a **generator object** that yields items one by one (lazy evaluation, memory efficient).

Syntax:

```
(expression for item in iterable if condition)
```

Examples:

1. Basic generator

```
gen = (x**2 for x in range(5))
print(gen)            # Output: <generator object ...>
print(list(gen))      # Output: [0, 1, 4, 9, 16]
```

Step-by-Step Explanation:

Step 1: `gen = (x**2 for x in range(5))`

This line defines a **generator expression**.

What is a Generator?

- A **generator** is like a list, but it **doesn't store values in memory**.
- Instead, it **yields values one at a time on demand** (i.e., lazily).
- It's **enclosed in parentheses** `()` instead of square brackets `[]`.

Let's break it down:

- `range(5)` generates the numbers: `[0, 1, 2, 3, 4]`
- `x**2` computes the **square** of each `x`
- So, `x**2 for x in range(5)` yields: `0, 1, 4, 9, 16`

But since it's a **generator**, it **doesn't compute all values immediately**.
It only computes each value when you **iterate** over it.

So at this point, `gen` is not the data itself — it's a **generator object**:

```
<generator object <genexpr> at memory_location>
```

Step 2: `print(gen)`

This prints the **generator object**, not its values:

```
print(gen)
# Output: <generator object <genexpr> at 0xXXXXXXXXXXXX>
```

This just tells you that `gen` is a **generator object**, located at some memory address.

Step 3: `print(list(gen))`

This line **converts the generator to a list** by **iterating** over it:

```
list(gen)
```

➞ This forces the generator to **start yielding values one by one** until it's exhausted:

Iteration	x	x**2	Yielded Value
1	0	0	0
2	1	1	1
3	2	4	4
4	3	9	9
5	4	16	16

So this becomes:

```
print(list(gen))
# Output: [0, 1, 4, 9, 16]
```

Important: Once a generator is consumed (fully iterated), it is **exhausted**.
If you try `list(gen)` again after this, it will return an **empty list**:

```
print(list(gen))  # Output: []
```

Why Use Generators?

Benefit	Description
✅ Memory efficient	Doesn't store all items in memory
✅ Lazy evaluation	Only computes values when needed
✅ Fast startup	No need to compute everything up front
✅ Clean syntax	One-liner for simple tasks

2. With filtering

```
gen = (x for x in range(10) if x % 2 == 0)
print(tuple(gen))     # Output: (0, 2, 4, 6, 8)
```

CLASSES AND OBJECTS

What is Object-Oriented Programming (OOP)?

It is a programming paradigm based on **objects** that contain **data (variables)** and **functions (methods)** together.

Key OOP Concepts:

- **Class** – Blueprint for creating objects
- **Object** – Instance of a class
- **Attributes** – Variables inside class
- **Methods** – Functions inside class
- **Constructor (`__init__`)** – Used to initialize object data

A. Defining a Class

```
class Student:
    # Constructor
    def __init__(self, name, roll):
        self.name = name
        self.roll = roll

    # Method
    def display(self):
        print("Name:", self.name)
        print("Roll:", self.roll)
```

B. Creating Objects

```
s1 = Student("Akhil", 101)
s2 = Student("Riya", 102)

s1.display()
s2.display()
```

Output:

```
Name: Akhil
Roll: 101
Name: Riya
```

```
Roll: 102
```

C. Types of Variables

Type	Description	Access
Instance Variable	Unique to each object	`self.name`
Class Variable	Shared by all objects	`Student.college`

```
class Student:
    college = "ABC College"  # class variable
    def __init__(self, name):
        self.name = name      # instance variable
```

D. Types of Methods

Method Type	Description	Example
Instance Method	Works on instance variables	`def display(self):`
Class Method	Works on class variables using `@classmethod`	`def change_college(cls):`
Static Method	General utility method using `@staticmethod`	`def greet():`

Example (All Methods):

```
class Student:
    college = "XYZ College"

    def __init__(self, name):
        self.name = name

    def show(self):
        print("Name:", self.name, "| College:", Student.college)

    @classmethod
    def change_college(cls, new_name):
        cls.college = new_name

    @staticmethod
    def greet():
        print("Welcome to Object-Oriented Programming in Python!")
```

E. Objects with Passed Values

```
s1 = Student("Mohit")
s2 = Student("Soniya")
Student.greet()
s1.show()
s2.show()
Student.change_college("DEF College")
s1.show()
```